SQL FOR DATA ANALYSIS · LESSON 18 OF 20

Business SQL: Cohorts, Funnels, Churn, Lifetime Value and Retention

Write the SQL behind the business questions that analysts are asked every week: cohort retention, conversion funnels, churn, customer lifetime value and retention rates, using the retail database.

Gap-fill lesson | 2026 | Advanced | 150 min

By the end of this lesson you can

- Build a cohort table and read a retention triangle
- Measure a multi-step funnel with conditional aggregation
- Define churn precisely and calculate a churn rate for a period
- Estimate customer lifetime value from historical orders
- Compute repeat-purchase and retention rates

Datasets used

`superstore_sales.csv`

Business definitions first

Most SQL errors in business reporting come from unclear definitions, not from syntax. Before you query, write the definition in one sentence:

- **Active customer**: placed at least one order in the period.
- **Churned customer**: active in the previous period and not active in this one.
- **Cohort**: customers grouped by the period of their first order.
- **Lifetime value (CLV)**: total gross profit or sales a customer generates over their relationship with the business, in this lesson measured on the data we have.

CASE STUDY **The retention review**

The growth team wants to know whether customers acquired in 2023 still buy in 2024 and 2025, and whether the business is losing customers faster than it gains them. The answers need a cohort table, a churn rate and a lifetime value estimate, all built from the same orders table.

Cohort retention

A cohort table counts how many customers from each first-order year were still active in later years. Each row is a cohort; each column is the year.

SQL

```sql
WITH first_order AS (
    SELECT customer_id, MIN(EXTRACT(YEAR FROM order_date)) AS cohort_year
    FROM orders
    GROUP BY customer_id
),
activity AS (
    SELECT DISTINCT o.customer_id, EXTRACT(YEAR FROM o.order_date) AS active_year
    FROM orders o
)
SELECT f.cohort_year,
       COUNT(DISTINCT f.customer_id)                                                AS
cohort_size,
       COUNT(DISTINCT CASE WHEN a.active_year = f.cohort_year     THEN a.customer_id END)
AS active_y0,
       COUNT(DISTINCT CASE WHEN a.active_year = f.cohort_year + 1 THEN a.customer_id END)
AS active_y1,
       COUNT(DISTINCT CASE WHEN a.active_year = f.cohort_year + 2 THEN a.customer_id END)
AS active_y2
FROM first_order f
LEFT JOIN activity a ON a.customer_id = f.customer_id
GROUP BY f.cohort_year
ORDER BY f.cohort_year;
```

RESULT

cohort_year	cohort_size	active_y0	active_y1	active_y2
2023	540	540	397	415
2024	193	193	156	0
2025	52	52	0	0

3 rows × 5 columns

Read the table across a row to see decay: if `active_y1` is well below `cohort_size`, the first renewal is where customers leave. Convert each count to a percentage of `cohort_size` for the retention triangle that stakeholders expect.

SQL

```sql
WITH first_order AS (
    SELECT customer_id, MIN(EXTRACT(YEAR FROM order_date)) AS cohort_year
    FROM orders GROUP BY customer_id
),
activity AS (
    SELECT DISTINCT customer_id, EXTRACT(YEAR FROM order_date) AS active_year FROM orders
),
cohort AS (
    SELECT f.cohort_year, a.active_year - f.cohort_year AS years_since_first,
COUNT(DISTINCT a.customer_id) AS active
    FROM first_order f
    JOIN activity a ON a.customer_id = f.customer_id
    GROUP BY 1, 2
),
sizes AS (
    SELECT cohort_year, COUNT(*) AS cohort_size FROM first_order GROUP BY cohort_year
)
SELECT c.cohort_year, c.years_since_first,
       ROUND(100.0 * c.active / s.cohort_size, 1) AS retention_pct
FROM cohort c
JOIN sizes s ON s.cohort_year = c.cohort_year
ORDER BY c.cohort_year, c.years_since_first;
```

RESULT

cohort_year	years_since_first	retention_pct
2023	0	100.0
2023	1	73.5
2023	2	76.9
2024	0	100.0
2024	1	80.8
2025	0	100.0

6 rows × 3 columns

::::tip Use the right denominator Retention is always measured against the cohort's starting size, not against the previous year's active count. Dividing by the wrong base is one of the most common cohort mistakes. ::::

Conversion funnels

A funnel counts customers who reach each stage. The retail data has no page views, so this example builds a simple funnel from the order history: customers who placed any order, customers who placed a second order, and customers who placed a third order.

SQL

```sql
WITH per_customer AS (
    SELECT customer_id,
           COUNT(DISTINCT order_id) AS orders_placed
    FROM orders
    GROUP BY customer_id
)
SELECT
    COUNT(*)                                                  AS stage_1_customers,
    COUNT(*) FILTER (WHERE orders_placed ⩾ 2)                 AS stage_2_repeat_buyers,
    COUNT(*) FILTER (WHERE orders_placed ⩾ 3)                 AS stage_3_loyal_buyers,
    ROUND(100.0 * COUNT(*) FILTER (WHERE orders_placed ⩾ 2) / COUNT(*), 1) AS
pct_stage_1_to_2,
    ROUND(100.0 * COUNT(*) FILTER (WHERE orders_placed ⩾ 3)
          / NULLIF(COUNT(*) FILTER (WHERE orders_placed ⩾ 2), 0), 1)       AS
pct_stage_2_to_3
FROM per_customer;
```

RESULT

stage_1_customers	stage_2_repeat_buyers	stage_3_loyal_buyers	pct_stage_1_to_2	pct_s
785	729	606	92.9	83.1

1 rows × 5 columns

Each stage is a subset of the one above it, so the counts can only fall. The conversion between stages is the figure to track: a small improvement from stage 1 to stage 2 usually matters more than any change further down.

::::note Funnels need ordered events A real funnel uses ordered events with timestamps (visit, add to cart, checkout, purchase). The pattern above is the same, but each stage must also be restricted to happen before the next, for example with `MIN(event_time)` per stage per user. ::::

Churn

Churn is easy to misreport, so define it in the query. The example below defines a customer as churned in a year if they were active the previous year and did not order in this year.

SQL

```sql
WITH active AS (
    SELECT DISTINCT customer_id, EXTRACT(YEAR FROM order_date) AS y FROM orders
),
years AS (SELECT DISTINCT EXTRACT(YEAR FROM order_date) AS y FROM orders)
SELECT p.y AS year,
       COUNT(*) AS active_last_year,
       COUNT(*) FILTER (WHERE c.customer_id IS NULL) AS churned_this_year,
       ROUND(100.0 * COUNT(*) FILTER (WHERE c.customer_id IS NULL) / COUNT(*), 1) AS
churn_rate_pct
FROM active p
LEFT JOIN active c ON c.customer_id = p.customer_id AND c.y = p.y + 1
WHERE p.y + 1 IN (SELECT y FROM years)
GROUP BY p.y
ORDER BY p.y;
```

RESULT

year	active_last_year	churned_this_year	churn_rate_pct
2023	540	143	26.5
2024	590	123	20.8

2 rows × 4 columns

Read the output carefully: the rate is churn from one year to the next, measured on customers who were active in the earlier year. Customers who were new in the later year are not part of this calculation.

::::warn Churn depends on the window A customer who bought in December 2023 and again in January 2025 is churned in 2024 under the yearly definition used above, then reactivated in 2025. Monthly, quarterly and yearly definitions give different churn rates. State the one you used on every chart. ::::

Customer lifetime value

A simple, honest estimate of customer lifetime value from history is the average gross profit per customer multiplied by the average number of orders per customer, with the period stated. Real CLV models add discounting and predicted future behaviour; this query gives the historical baseline.

SQL

```sql
WITH cust AS (
    SELECT o.customer_id,
           COUNT(DISTINCT o.order_id) AS orders,
           SUM(i.sales)               AS revenue,
           SUM(i.profit)              AS gross_profit
    FROM orders o
    JOIN order_items i ON i.order_id = o.order_id
    GROUP BY o.customer_id
)
SELECT COUNT(*)                              AS customers,
       ROUND(AVG(orders), 2)                 AS avg_orders,
       ROUND(AVG(revenue), 0)                AS avg_revenue,
       ROUND(AVG(gross_profit), 0)           AS avg_profit_to_date,
       ROUND(AVG(gross_profit / NULLIF(orders, 0)), 0) AS avg_profit_per_order
FROM cust;
```

RESULT

customers	avg_orders	avg_revenue	avg_profit_to_date	avg_profit_per_order
785	4.08	256,578	23,379	5,881

1 rows × 5 columns

To find the value of a segment, repeat the query per segment (for example, by `segment` from the customers table). Compare segments on profit, not revenue: a segment with high revenue and thin margins may be worth less.

Retention and repeat rate

The repeat rate is the share of customers with two or more orders. It is simple and widely reported, and it shows whether one-time buyers are the main pattern.

SQL

```sql
WITH counts AS (
    SELECT customer_id, COUNT(DISTINCT order_id) AS orders FROM orders GROUP BY customer_id
)
SELECT COUNT(*)                                                  AS customers,
       COUNT(*) FILTER (WHERE orders = 1)                        AS one_time_buyers,
       COUNT(*) FILTER (WHERE orders ⩾ 2)                        AS repeat_buyers,
       ROUND(100.0 * COUNT(*) FILTER (WHERE orders ⩾ 2) / COUNT(*), 1) AS repeat_rate_pct
FROM counts;
```

RESULT

customers	one_time_buyers	repeat_buyers	repeat_rate_pct
785	56	729	92.9

1 rows × 4 columns

::::key What to remember - Write the definition (active, churned, cohort, CLV) before writing SQL. - Cohorts group customers by first purchase; retention divides by the cohort's starting size. - Funnels are subsets that shrink at each stage; report the stage-to-stage conversion. - Churn depends on the window you choose, so state it. - CLV from history is a baseline; use profit, not revenue, to compare segments. ::::

Quick quiz

5 questions · tap an answer to check it

1. In a cohort table, a customer's cohort is based on:

A Their most recent order

B Their first order

C Their segment

D The product they bought most

Answer: B. A cohort groups customers by the period of their first activity.

2. Retention for year 1 should be divided by:

A The previous year's active count

B Total revenue

C The cohort's starting size

D The number of orders

Answer: C. Retention is measured against the group you started with.

3. A funnel's stage 2 count is larger than stage 1. What is wrong?

A Nothing; funnels can grow

B Stage 2 is a subset of stage 1, so the definition or the join is wrong

C Stage 1 should be ignored

D Stage 2 must use revenue

Answer: B. Each stage must be a subset of the stage before it.

4. Why is comparing segments on profit safer than on revenue?

A Profit is always larger

B Revenue can be high while margins are thin, so profit shows which segment is worth more

C Revenue cannot be queried

D Profit includes discounts

Answer: B. Profit shows what the business keeps.

5. The repeat rate counts customers with:

A One order

B Two or more orders

C No orders

D The highest spend

Answer: B. Repeat buyers placed at least two orders.

EXERCISE **Win-back list**

Write a query that lists customers who were active in 2024 but placed no order in 2025. For each, show their segment, their 2024 gross profit and their last order date. Order by 2024 profit, highest first, and return the top 20.

Show solution (worked answer)

Hint: find 2024 customers and 2025 customers in two CTEs, keep 2024 customers with LEFT JOIN to 2025 and WHERE the 2025 side is NULL. Join `customers` for the segment and aggregate `order_items` for profit.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.